

1. Problem Statement

The goal of this project is to build an **Automated Essay Scoring (AES)** system for student essays using classical machine learning. We work with **Essay Set 1** from the ASAP (Automated Student Assessment Prize) dataset, where each essay receives a **holistic score between 2 and 12**.

Our objectives are:

- to design **interpretable linguistic features** that capture essay quality,
 - to train and compare several machine learning models,
 - and to evaluate them using **Quadratic Weighted Kappa (QWK)** and other metrics, then generate predicted scores for unseen essays.
-

2. Dataset and Preprocessing

We use the publicly available ASAP dataset (Kaggle), which originally contains:

- `essay_id`, `essay_set`
- `essay` (raw text)
- human scores such as `domain1_score` (holistic score for Essay Set 1)

The original dataset does **not** contain ready-made features. We therefore:

1. Downloaded the official train/validation/test splits from the competition repository.
2. Ran a feature-extraction notebook (in Google Colab) to compute additional features from the essay texts.
3. Saved three new CSV files with features:
 - `training_full_features.csv`
 - `valid_full_features.csv`
 - `test_full_features.csv`

All three files live under:

```
/content/drive/MyDrive/emperical/dataset New/
```

In the notebook, we **restrict ourselves to Essay Set 1**:

- `df_train1`: all training essays where `essay_set == 1` ($n = 1,783$)
- `df_valid1`: all validation essays where `essay_set == 1` ($n \approx 589$)
- `df_test1`: all test essays where `essay_set == 1` ($n \approx 594$)

The **target label** for training is:

- `y = df_train1["domain1_score"].astype(int)` (scores $\in \{2, \dots, 12\}$)

For modeling, we use the following **9 engineered features** (columns) from each essay:

1. Word Count
2. Sentence Count
3. Avg. Sentence Length
4. Spelling Error Count
5. Grammar Error Count (NaNs filled with 0 in the full run)
6. Type-Token Ratio (TTR)
7. Stopword Ratio
8. Use of Academic Words
9. Similarity (similarity to the prompt / reference text)

Feature matrices:

- $x_{\text{train}} \in \mathbb{R}^{(1783 \times 9)}$ for training,
- x_{valid} for validation essays (no labels),
- x_{test} for test essays (no labels).

3. Feature Engineering (Summary)

The feature extraction code (run before the ML pipeline) uses standard NLP tools (tokenization, stopword lists, academic word lists, and spell-checking) to compute:

- **Length and structure:**
 - total word count, sentence count, average sentence length.
- **Surface correctness:**
 - spelling error count (via spell checker),
 - grammar error placeholder (initially designed to use a grammar checker, but in the final full run this was turned off for runtime reasons and filled with 0).
- **Lexical diversity and style:**
 - type–token ratio (TTR) for vocabulary richness,
 - stopwords ratio,
 - proportion of academic words (AWL).
- **Content relevance:**
 - similarity between essay text and the prompt/reference (e.g., cosine similarity on vectorized representations).

These features are intentionally **interpretable**, so we can relate model decisions back to properties of the writing.

4. Models and Training Setup

We experimented with both **classification** and **regression** models:

4.1 Classification Models

These predict the score as a **discrete label** (2–12):

- **LogisticRegression_all**
 - Multinomial logistic regression with L2 regularization.
 - Wrapped in a Pipeline with StandardScaler.
- **RandomForestClassifier_all**
 - Random Forest with ~400 trees, moderate regularization (`min_samples_split`, `min_samples_leaf`).
- **LinearSVM_all**
 - Linear Support Vector Machine (`LinearSVC`) with `class_weight="balanced"` to handle rare scores, also wrapped in a Pipeline with StandardScaler.

4.2 Regression Models

These treat the score as **continuous** and then round to an integer:

- **RandomForestRegressor_all**
 - Random Forest regressor with ~400 trees.
- **GradientBoostingRegressor_all**
 - Gradient boosting regressor (scikit-learn) with learning rate 0.05 and 400 estimators.
- **XGBRegressor_all**
 - XGBoost regressor (tree-based gradient boosting), tuned with:
 - `n_estimators = 600`,
 - `max_depth = 4`,
 - `learning_rate = 0.05`,
 - `subsample = 0.8`,
 - `colsample_bytree = 0.8`.

For regression models, predictions are post-processed with:

- rounding to nearest integer,
- clipping between 2 and 12,

before computing classification metrics.

5. Evaluation Metrics and Procedure

5.1 Metrics

We evaluate on the **training set (Essay Set 1)** using 5-fold cross-validation:

- **Accuracy:** fraction of exactly correct scores.
- **Macro F1-score:** average F1 across all score levels (treating each score as a class).
- **Quadratic Weighted Kappa (QWK):**
The main metric in AES competitions, measuring agreement between human scores and model predictions while penalizing large errors more than small ones (e.g., predicting 7 instead of 8 is less serious than predicting 3 instead of 8).
- **RMSE (for regressors):** root mean squared error on the **continuous** predictions before rounding.

Because scores are **ordinal** and near misses (± 1 point) are still acceptable in grading, **QWK** is the primary metric we use to compare models.

5.2 Cross-Validation

We use **Stratified 5-fold** cross-validation on the 1,783 training essays of Set 1:

- Each model is trained 5 times; in each fold:
 - 80% essays are used for training,
 - 20% for validation.
- We record Accuracy, Macro F1, QWK (and RMSE for regressors) for each fold and report the **mean $\pm$ standard deviation**.

We then pick the **best model** based on **mean QWK** across folds.

6. Results

6.1 Model Comparison (QWK and Other Metrics)

The cross-validation results show:

- Among **classification models**:
 - RandomForestClassifier_all reaches QWK ≈ 0.79 .
 - LogisticRegression_all and LinearSVM_all perform slightly lower (QWK ≈ 0.77 – 0.78).
- Among **regression models** (rounded to integer scores):
 - RandomForestRegressor_all is the best overall:
 - **Mean QWK ≈ 0.83**
 - Mean Accuracy ≈ 0.51
 - Mean Macro F1 ≈ 0.42
 - Mean RMSE ≈ 0.80
 - GradientBoostingRegressor_all achieves QWK ≈ 0.82 .

- `XGBRegressor_all` achieves $\text{QWK} \approx 0.81$.

Overall, **RandomForestRegressor_all** achieves the **highest QWK** and is selected as the **best model**. A QWK around 0.83 is considered *strong* agreement in essay scoring (close to human–human agreement), especially given only 9 handcrafted features and 11 possible score levels.

6.2 Hold-out Visualization (Internal Only)

To visualize how the best model behaves, we also perform a single 80/20 split of the training data (for plots only):

- Fit `RandomForestRegressor_all` on 80% of the essays.
- Evaluate on the remaining 20% (hold-out set).

On this hold-out, we obtain:

- Accuracy ≈ 0.52
- Macro F1 ≈ 0.44
- QWK ≈ 0.83

We generate:

- an **error distribution plot** showing that most predictions are within ± 1 point of the true score,
- and a **confusion-matrix-style heatmap**, where most mass lies along the diagonal, confirming that the model rarely makes large scoring mistakes.

7. Final Model and Predictions on Validation/Test

After model selection, we **refit the best model** (`RandomForestRegressor_all`) on all 1,783 training essays for Essay Set 1:

```
best_model.fit(X_train, y)
```

Then we apply this fitted model to:

- `x_valid` (features from `valid_full_features.csv` for Set 1),
- `x_test` (features from `test_full_features.csv` for Set 1).

Because the validation and test feature files do **not** contain `domain1_score`, we **cannot compute QWK** on them. Instead, we generate predictions:

- `valid_predictions_set1_best.csv`
- `test_predictions_set1_best.csv`

Each file contains:

- `essay_id`
- `essay_set`
- `predicted_domain1_score`

These predictions can be used to:

- inspect qualitative examples (essay text + predicted score),
- or compare to human scores later if ground truth labels become available.

8. Discussion and Future Work

With a relatively small set of **9 handcrafted features**, the best model achieves a **QWK of about 0.83** on Essay Set 1, which is quite strong given the difficulty of predicting 11 discrete score levels (2–12) from short essays.

However, there are several ways to improve:

1. **Richer Features**
 - Character and word n-grams (TF-IDF),
 - Part-of-speech tag statistics,
 - Cohesion/coherence measures,
 - Deep semantic embeddings from transformer models.
2. **More Sophisticated Models**
 - Larger gradient boosting models or tuned XGBoost,
 - Neural networks using pre-trained language models.
3. **Better Error Analysis**
 - Analyze which score ranges or essay types are hardest,
 - Inspect essays where the model is off by ≥ 2 points.

Despite these limitations, the current system already demonstrates that:

- simple, interpretable features,
- combined with classical ML (especially Random Forest regression),

can yield **competitive scoring performance** on the ASAP Essay Set 1, making this a solid baseline and a good proof-of-concept for automated essay scoring in our course project.